

Coding Agent 验证行为实验任务 集调研

横纵分析法深度研究报告

研究时间：2026-08-15 | 所属领域：AI Coding Agent × Visual Analytics × Software Testing | 研究

对象类型：实验任务集与评测协议

作者: Codex

研究时间：2026-08-15 | 所属领域：AI Coding Agent × Visual Analytics × Software Testing | 研究对象类型：实验任务集与评测协议

一、一句话定义

这不是再造一个“谁修复 issue 更多”的 Coding Agent 排行榜，而是构造一组能够稳定诱发、完整记录并可靠裁决验证行为的任务，让研究者看见不同 Harness 如何在代码修改之后选择测试、解释失败、扩大范围、重复执行以及决定停止。

二、结论先行：不要重新选 30 个任务

本次调研最重要的发现不在外部，而在已有的 TestGuard benchmark 资产。其 1.2.0 预注册协议固定了三个公开数据源、两种语言生态、三类项目、三个任务难度、四个风险档位，以及 18 个真实 issue 加 6 个受控维护任务的平衡 pilot。任务 manifest 当前包含 34 条记录，覆盖 19 个仓库标识；其中真正用于平衡 pilot 的是 18 个公开任务和 6 个本地受控任务。

因此，原先“先收集 30 个 baseline，再收集 30 个 shadow”的方案应该停止扩张。它只规定了数量，没有控制项目类型、难度、语言、验证成本和任务风险。TestGuard 已经把这些变量拆开，而且保留了固定 seed、数据 revision、base commit、gold patch 摘要、测试 oracle 和候补任务。重新选一套任务不仅浪费已有工作，还会引入在看到结果后挑样本的嫌疑。

推荐采用下面的四阶段结构：

阶段	任务	Harness	重复	轨迹数量	允许回答的问题
P0 采集链路校准	6 个受控任务	3	2	36	事件是否完整、命令归一化是否正确、可视化是否可读
P1 平衡 pilot	24 个任务 (18 公开 + 6 受控)	3	3	216	哪些冗余验证模式稳定出现；不同 Harness 有何描述性差异
P2 论文主语种	30 个任务	3	3	270	多语言跨 Harness 模式是否稳定；支撑可视分析与用户研究
P3 强确认性研究	至少 48 个任务	3	5	720	干预是否具有跨项目因果效果与非劣安全性

P3 的 720 条轨迹成本很高，不应在界面、事件 schema 和标注 rubric 尚未稳定时直接启动。论文最合理的推进方式是先完成 P0，再用 P1 形成可视分析系统和初步用户研究；随后以 10 个 SWE-bench Verified 与 20 个 SWE-PolyBench Verified 任务组成 270 条正式主语料。只有在论文需要声称自动干预具有稳定因果效果时，才启动 P3。

本次建议的工作名是 **VerifyTrace**：它不是新的修复能力 benchmark，而是从固定软件工程任务中派生出的“验证行为轨迹数据集”。原有 `TestGuard` 负责风险策略和执行约束，`ai-coding-agent-test-strategy` 负责研究叙事与可视分析。二者后续应明确主从关系，避免维护两份相互漂移的规则。

三、纵向分析：Coding Agent 任务集为什么走到今天

3.1 第一阶段：测试只是终点裁判

早期代码生成评测把程序看成一个输入输出函数。HumanEval、MBPP、LiveCodeBench 一类任务给出函数说明和若干测试，模型生成代码，运行器最后计算 `pass rate`。这个设计干净、便宜、易于复现，也适合比较模型的局部代码能力。

但它几乎看不到软件开发过程。模型是否读了错误文件、是否运行了十次全量测试、是否在失败后新增了无意义 mock，都不会改变最终的 `pass/fail` 结构。即使引入多轮反馈，任务仍然以孤立函数为中心，缺少真实仓库的依赖关系、构建系统、历史约束和测试拓扑。

对 VerifyTrace 来说，这类任务只能用来校准事件采集，不能承担主研究结论。它们的测试入口太短，Agent 没有多少“选择测试”的空间；如果把它们当主样本，很可能得到“所有 Harness 都只跑一条命令”的平坦轨迹。

3.2 第二阶段：SWE-bench 把真实仓库和 issue 带进评测

SWE-bench 的关键变化，是把任务单位从函数改成真实 GitHub issue。每条任务固定仓库、base commit、问题描述和测试 oracle，Agent 必须浏览代码、修改多个文件并在项目环境中验证。测试不再只是终点裁判，也成为 Agent 在修复过程中的反馈渠道。

这个变化让验证行为第一次具有研究价值。同一个 issue 可以触发多种路径：有的 Agent 先定位目标测试，有的先跑完整套件；有的根据失败缩小范围，有的把环境错误当成代码错误继续扩张。最终补丁都正确，不代表过程成本相同。

SWE-bench 的代价也很明确。真实项目依赖复杂，容器体积大，历史 commit 可能依赖过期包源或架构；任务可运行性本身会吞掉大量工程时间。公开 issue 和 gold patch 还存在训练污染风险。SWE-bench 适合作为任务来源，但不能把官方 leaderboard 的 resolved 率直接搬来解释测试效率。

3.3 第三阶段：Verified、Multilingual 和 Pro 开始修复代表性问题

SWE-bench Verified 通过人工核验缩小为 500 个更可靠的任务，并提供更清晰的难度信息。它减少了错误描述、不可复现环境和不公平 oracle，但主要生态仍是 Python。对于要研究测试命令选择的工作，Verified 的价值不只是“更可信”，还在于它包含 `FAIL_TO_PASS` 和 `PASS_TO_PASS`，可以把“少跑测试”与“没有漏掉缺陷”同时纳入评估。

Verified 仍然不能把官方测试变成绝对真值。SWE-Bench+ 对成功案例的审计指出，部分 issue 已在讨论中直接暴露方案，部分测试不足以排除可疑补丁；UTBoost 也报告了测试不足和错误通过案例。这两项工作共同说明：任务集必须把官方 F2P/P2P、隐藏补充测试、mutation/seeded fault 和人工复核分层使用，不能把一个 leaderboard 的 resolved 字段直接翻译为“质量没有下降”。

Multi-SWE-bench 与 Flash 子集进一步扩展语言和仓库。TestGuard 当前固定使用 SWE-bench Verified 的 Python 任务，以及 Multi-SWE-bench Flash/完整集中的 JavaScript、TypeScript 任务。这是一个务实组合：Pytest 与 Jest/Vitest/Mocha 形成两类差异明显、又能在本机和容器中结构化观察的测试生态。

SWE-bench Pro、FeatureBench、Terminal-Bench 等后续工作继续把任务推向更长周期、更复杂功能和更宽终端环境。它们提升了真实度，却未必适合第一版 VerifyTrace。一个需要数小时、跨十几个文件、包含外部服务的任务，虽然更接近真实工程，却会把研究预算消耗在环境恢复和任务成功率上。当多数 Agent 根本无法完成任务时，研究者看到的主要是失败，而不是细粒度验证策略差异。

3.4 第四阶段：从结果 benchmark 转向过程与 loop

2025 年的 SeaView 已经指出 SWE Agent 轨迹很长、难以定位错误，也难以比较不同模型和超参数运行；其核心贡献是帮助研究者检查和对比轨迹。2026 年的 AgentGUI 进一步把观察和干预合并，并报告用户识别轨迹关键信息所需时间下降。可视化不再只是日志皮肤，而成为人类监督长任务 Agent 的接口。

LoopsBench 把关注点从一次性 issue 修复推进到长期 loop：任务由可分别测试的开发单元 DAG 构成，已完成节点继续承担回归义务。它揭示一个重要趋势：未来 Agent 的验证问题不只是“这次跑多少测试”，还包括“过去已经完成的行为，什么时候需要重新验证”。

同一时期，`Same Task, Different Work: Prompt-Induced Waste in Coding Agents` 用 4,644 次运行直接测量相同正确结果背后的工作浪费。论文报告，高冗余验证运行的成本可达干净运行中位数的 18 倍，工具调用约 2.5 倍，耗时约 3 倍，却没有形成成功率梯度。它已经证明“验证浪费存在且重要”，也意味着 VerifyTrace 不能把“发现 Agent 会多测”当成论文创新。

真正尚未被充分解决的是：这些浪费从轨迹的哪一个决策点出现，它们与代码变更、依赖范围、失败类型和 Harness 规则之间是什么关系，用户如何通过可视分析形成可执行的干预判断。

3.5 第五阶段：任务集需要从 **outcome-first** 变成 **trace-first**

传统 benchmark 的最小数据单位是“任务 + 最终补丁 + 最终分数”。VerifyTrace 的最小数据单位应改成“任务 + 运行配置 + 有序事件 + 决策证据 + 最终 oracle”。任务仍来自 SWE-bench 等成熟来源，但研究数据不再止于 resolved。

这要求任务集具备五个属性：

1. 存在选择空间：仓库同时有定向测试、包级测试和全量测试，Agent 能表现出不同验证路径。
2. 存在可判定风险：变更能够归入 off、smoke、standard、thorough，而不是每条任务都天然必须全量验证。
3. 存在可靠 **oracle**：最终正确性不能由 Agent 或同一模型自评，必须有隐藏回归测试、全量测试或 seeded fault/mutation 证据。
4. 存在可重复环境：每次运行能从同一 base commit、lockfile 和依赖缓存开始，环境失败可与代码失败分开。
5. 存在足够异质性：语言、项目类型、任务难度和全量测试成本不能完全重合，否则 Harness 差异会被仓库差异冒充。

TestGuard 1.2.0 已经覆盖其中大部分。下一步不是再扩大候选池，而是把“验证路径是否可观察”加入任务准入条件。

四、横向分析：候选任务来源适不适合 VerifyTrace

4.1 对比维度

本调研不按 leaderboard 热度选择数据集，而按论文问题反推任务来源。八个关键维度如下：

- 是否固定真实仓库和 base commit；
- 是否提供可执行的最终 oracle；
- 是否区分缺陷修复测试与原有回归测试；
- 是否允许 Agent 自主选择测试范围；
- 是否覆盖多语言和不同测试拓扑；
- 是否支持同一任务跨 Harness 重放；
- 是否能控制公开答案污染；
- 环境成本是否适合先做 pilot。

4.2 主要候选对比

来源	真实仓库	Oracle	多语言	验证选择空间	环境成本	主要用途	不适合作为唯一来源的原因
SWE-bench Verified	是	强，F2P/P2P	主要 Python	高	高	Python 真实 issue 主样本	语言单一、公开污染、容器重
SWE-bench Live	是，较新 issue	SWE-bench 式	多仓库	高	高	污染敏感性与近期任务	仍是公开数据，oracle 需抽查
Multi-SWE-bench Flash	是	强，fix/test patch	是	高	中到高	JS/TS 真实 issue 主样本	Flash 任务分布可能偏向可运行样本
Multi-SWE-bench 完整集	是	强	是	高	高	补齐矩阵缺格	难度标签和环境一致性不如精炼子集
SWE-PolyBench Verified	是	强，冻结容器与原始测试日志	Java/JS/TS/Python	高	高	正式多语言主样本	公开污染；需冻结 revision 并重新统计任务分布
SWT-Bench	是	测试须在缺陷版失败、修复版通过	Python	聚焦测试生成	高	测试价值辅助研究	目标是写测试，不等于修复过程中过度验证
SWE-smith	合成于真实仓库	可规模化生成	以 Python 为主	中到高	高	扩大训练或敏感性分析	合成任务与真实开发意图仍有距离

来源	真实仓库	Oracle	多语言	验证选择空间	环境成本	主要用途	不适合作为唯一来源的原因
LoopsBench	是/来源可追溯	开发单元级测试	多语言	很高	很高	长周期回归行为扩展	首版成本过大，任务成功率可能过低
FeatureBench / 长功能任务	是	功能级	视数据集而定	很高	很高	后续检验长期迭代	变量太多，不适合先校准冗余判定
本地受控维护任务	部分模拟	完全可控	JS + Python	人工可设计	低	事件、规则和界面校准	不能证明真实项目泛化
未公开 maintainer 任务	是	可由维护者提供	可选	高	中	污染控制与外部效率	获取和双盲管理成本高

没有一个公开 benchmark 单独满足全部条件。最合理的结构不是选出“唯一赢家”，而是分层组合：受控任务保证因果和调试，Verified/Multi-SWE 保证真实度，held-out 任务控制污染，LoopsBench 只作为后续长期回归扩展。

SWE-PolyBench Verified 值得作为正式主语料的新候选。它提供 Java、JavaScript、TypeScript、Python 的 instance-level 冻结环境，以及 passing/failing tests 和原始测试日志，适合验证“冗余测试是否只是 Pytest 生态现象”。其 README 的 Verified 总数与分语言数字存在轻微不一致，因此正式使用时必须固定 dataset revision，并从 manifest 重新统计，不能照抄首页数字。

4.3 SWE-bench Verified：最可靠的 Python 主干

Verified 最适合承担 standard/thorough 风险任务。固定 base commit 与 F2P/P2P 使最终正确性有明确边界；人工难度标签还能避免用补丁行数直接冒充任务难度。

它的弱点同样需要写入方案。第一，Agent 可能在训练语料或公开讨论中见过 issue 与修复。第二，完整容器环境对磁盘、架构和依赖下载要求高。第三，Python 项目的测试发现和选择机制较统一，如果只使用 Verified，很难判断结果能否迁移到 npm workspace、monorepo 和前端构建链。

TestGuard 已固定 Verified revision `91aa3ed51b709be6457e12d00300a6a596d4c6a3`，并从中选择不同难度、项目类型和规模任务。这个 revision 应继续冻结，不能为了“更好跑”随意跟随 latest。

4.4 Multi-SWE-bench：补齐 JS/TS 与复杂 workspace

Multi-SWE-bench 的价值不只是多一种语言。JavaScript/TypeScript 仓库常同时存在 format、lint、typecheck、单包测试、集成测试和构建命令，恰好提供验证决策空间。Vue、Material UI、Darkreader 等项目还能覆盖 monorepo、UI 和应用型验证拓扑。

TestGuard 采用 Flash 作为可行性主来源，再用完整集补齐 18 格矩阵，是合理的分层策略。需要保留的风险有两点：一是不同仓库的包管理器和历史 Node 版本差异很大；二是某些任务虽然 gold patch 很小，全量构建却可能极慢。报告时必须把生产代码规模、项目类型和全量测试耗时分开，不能把“大仓库”等同于“难任务”。

4.5 受控维护任务：不是玩具，而是测量仪器校准件

本地 6 个零依赖任务覆盖 JS/Python 的 off 与 smoke 风险，已经完成 AB/BA 顺序、日志 hash、隔离和 seeded fault 校准。它们不能证明产品在真实项目中提速，但有一个公开 issue 数据集无法替代的作用：当事件缺失、命令归一化错误或可视化误导时，研究者知道预期轨迹应该是什么。

这类任务应保留在每次 schema 或采集器升级的回归套件中。它们不是论文主样本，却是保证论文数据不是采集 bug 的基础设施。

4.6 LoopsBench 与长功能任务：适合第二篇，不适合第一版

LoopsBench 的 DAG 与回归义务非常适合研究“验证债务如何随长期开发累积”。它甚至可能比单 issue 更接近未来 Coding Agent 的真实工作方式。但把它放进第一版会同时引入任务规划、需求分解、上下文压缩、长期记忆和多轮回归等变量。

第一篇论文需要先把单任务验证 loop 看清楚。等 VerifyTrace 能稳定表示“变更 -> 测试 -> 失败 -> 重试 -> 停止”后，再扩展为跨开发单元的回归义务图。这个顺序不是保守，而是保证每篇工作都有可检验的中心问题。

4.7 Harbor 与 ATIF：它不是任务集，却可能省掉一半适配工作

跨 Harness 研究最容易掉进“三套私有日志分别解析”的陷阱。Harbor 已提供面向终端 Agent benchmark 的统一运行层，并以 Agent Trajectory Interchange Format (ATIF) 记录消息、工具调用、观察、token 与 cost。Codex、OpenCode 已有适配基础，Maka 需要新增 adapter。

建议把 Harbor 当中立运行层，而不是数据集来源；任务仍来自冻结的 TestGuard/SWE-bench/PolyBench manifest。容器内再增加独立命令观察层，记录 `execve` /PTY、进程树、工作目录、退出码、耗时和 diff hash。这样即使某个 Harness 自报日志更详细，也不会因此显得“行为更多”。

引入 Harbor 前仍要验证两点：它的 adapter 没有替 Harness 改写系统 Prompt、工具权限和停止行为；ATIF 对文件变更和测试子进程的粒度足够，不够的字段由独立观察层补充，不能用模型推测。

五、现场审计：现有 TestGuard 任务集已经做到什么

5.1 已有资产

当前本地协议已经具备以下可发表研究需要的基础：

- 协议版本 1.2.0，在 A/B 结果前冻结任务和选择规则；
- 固定随机种子 `20260809`；
- SWE-bench Verified、Multi-SWE-bench Flash 和完整集的精确 revision；
- 12 个 feasibility 公开任务，覆盖 2 个生态、9 个仓库、三个难度；
- 18 个 balanced 公开任务，覆盖 `2 生态 × 3 项目类型 × 3 难度`；
- 6 个受控 off/smoke 任务；
- 固定 base commit、项目规模探测、测试 runner 和项目类型证据；
- gold patch 只用于分层与最终评分，不提供给 Agent；
- 明确禁止把重复命令次数冒充任务样本；
- 任务内配对、分层 bootstrap、timeout 保留、基础设施失败单列；
- F2P、P2P、最终 full 与 mutation/seeded fault 的正确性闸门；
- 低价值测试采用盲化双评审，而不是作者单人判断。

这套设计已经超过“随便抽 30 个 issue”的水平。它应成为新研究的底座。

5.2 当前没有完成的部分

`check-design.mjs` 的 fresh 输出显示：

- `harness_feasibility=true`；

- 18 格矩阵没有缺格；
- 规模与项目类型没有完全混杂；
- 6 个受控任务齐全；
- `balanced_pilot=false`；
- 直接原因是 `sphinx-doc__sphinx-8120` 和 `expressjs__express-3870` 仍标记为需要第二评审。

Sphinx 同时提供库接口与文档构建 CLI，Express 名称像框架/服务但固定版本实际以被应用导入的公共 API 为主。这两个边界案例需要独立评审者在不知道实验结果的情况下查看固定 commit 的 README、package manifest、入口和测试拓扑。评审不同意时应保留分歧，而不是为了补满矩阵强行改标签。

现有协议的另一个阻塞已经变化。2026-08-09 文档记录本机仅有约 3.8 GiB 可用空间；2026-08-15 fresh check 显示约 175 GiB 可用，已经超过 SWE-bench 官方常见的约 120 GiB 建议。磁盘不再是当前首要阻塞。真正剩余的问题是本机 Docker server 为 `linux/arm64`，某些官方镜像和历史依赖在 Apple Silicon 上可能需要 x86_64 模拟或独立 Linux x86_64 环境。

DMIT VPS 是 Debian x86_64，但只有 1 vCPU、约 1.9 GiB RAM 和 13 GiB 可用磁盘，不适合承载 SWE-bench 容器评测。它可以承担轻量事件汇总、报告服务或调度入口，不能承担主实验执行。

5.3 现有协议与新论文问题不完全一致

TestGuard 当前端到端实验比较的是同一 Agent 下的三种治理条件：default、policy-only、testguard-enforced。新的可视化研究已经决定先比较 Codex、Maka、OpenCode 三个 Harness，并固定模型、任务、Prompt、权限和预算。

如果同时把“Harness 三水平”和“治理三条件”放进第一轮，24 个任务、3 个 Harness、3 个条件、每条件 3 次就需要 648 次运行。界面和标注尚未稳定时，这个规模没有意义。

因此应拆成两个研究阶段：

1. 观察性跨 **Harness** 数据集：只跑各 Harness 的冻结 baseline，形成 216 条平衡 pilot 轨迹，用于建立冗余模式 taxonomy 和可视分析任务。
2. 干预性子实验：在观察性数据冻结后，从各分层预先抽取任务，比较 baseline 与 advisory/limited-enforcement；只有这一步才能声称干预导致提速。

第一阶段可以描述相关性和差异，不能写“某个 Harness 导致浪费”；第二阶段随机化并控制其他变量后，才允许因果表述。

六、为可视化论文重新定义任务集

6.1 任务不是 issue，而是可重放实验包

每个 VerifyTrace 任务包至少包含：

```
task_id
dataset + revision
repo + base_commit
problem_statement
language + project_type + project_size
task_difficulty + risk_tier
runtime + lockfile + dependency_cache_key
allowed_tools + wall_time_budget
visible_test_entrypoints
hidden FAIL_TO_PASS / PASS_TO_PASS oracle
final_full_command
gold_patch_hash ( 不向 Agent 暴露正文 )
selection_reason + exclusion_policy
```

同一个包必须能为 Codex、Maka、OpenCode 创建等价的新工作树。三者使用同一模型版本、温度/推理配置、Prompt、网络策略和权限；如果某 Harness 不支持同一种推理参数，应记录为结构差异，而不是静默使用默认值。

6.2 增加“验证可观测性”准入条件

原协议关注任务是否有 patch 和 oracle，新论文还应在看到 Agent 结果之前检查：

- 仓库是否提供至少两个粒度不同的验证入口；
- test runner 是否能输出机器可解析的目标与结果；
- full suite 是否能在预算内完成，或能明确标记异步验证；
- 环境安装失败是否可稳定复现并分类；
- 测试命令能否归一化为 canonical id；
- changed files 能否映射到 package/workspace/test target；
- 任务是否天然只允许唯一测试路径。

如果一个任务只有 `npm test`，且没有任何定向入口，它仍可用于成功率 benchmark，却不适合研究“为什么 Agent 选择了过大的测试范围”。

6.3 轨迹事件 schema

可视化不能直接依赖三个 Harness 各自的原始日志。建议先转成统一事件流：

```
{
  "schema_version": 1,
  "run_id": "...",
  "task_id": "...",
  "harness": "codexlmakalopencode",
  "model": "fixed-model-version",
  "event_index": 42,
  "timestamp": "ISO-8601",
  "phase": "inspect|edit|verify|diagnose|repair|finalize",
  "event_type": "file_read|file_write|test_plan|command|test_result|failure_class|stop",
  "changed_files_since_last_verify": [...],
  "canonical_command_id": "...",
  "test_scope": "target|package|affected|full",
  "duration_ms": 12000,
  "exit_code": 1,
  "failure_signature": "hash",
  "risk_tier": "standard",
  "decision_evidence": ["changed package X", "previous failure Y"],
  "raw_event_ref": "content-addressed pointer"
}
```

不要依赖隐藏 chain-of-thought。三套产品对推理文本的暴露程度不同，强行比较会形成不可控缺失。可以要求 Agent 在执行高成本验证前输出结构化、简短的 `test_plan`，但这本身会改变行为，应作为统一实验 Prompt 的一部分，并在所有 Harness 中一致应用。

6.4 冗余验证 taxonomy

“蠢测试”不能作为论文标签。建议把它拆成可观察类型：

1. **Exact repeat**：canonical command 相同，相关代码和测试未变化，再次执行。
2. **Equivalent repeat**：入口不同但解析后目标集合、环境和语义相同。
3. **Unattributed retry**：失败签名相同，Agent 未完成代码/测试/环境/flaky 分类便重跑。
4. **Scope overshoot**：可靠影响映射只涉及叶子模块，却提前执行 package/full；是否冗余需结合风险人工裁决。
5. **Premature full**：定向反馈尚未获取就运行高成本全量验证。

6. **Test expansion without contract**：新增测试无法对应公共行为、回归缺陷或 mutation。

7. **Necessary revalidation**：看似重复，但相关代码、fixture、配置或环境已经变化；必须明确标为非冗余。

前两类可以确定性标注；第三类需要失败分类；第四至第六类由受约束模型读取 diff、依赖图、历史结果和 rubric 进行预标注，低置信度或高风险样本交给人类。论文 ground truth 不能由同类模型直接定案。

建议两名评审者对模糊样本盲化独立标注，报告 Cohen's kappa 或 Krippendorff's alpha，并保留 disagreement。模型裁判的准确率应相对人类 adjudication 单独报告，而不是混进系统效果。

6.5 可视分析任务

任务集应服务于用户问题，而不是服务于图表数量。首版系统至少支持四个分析任务：

- **T1 定位起点**：找到一条轨迹中第一次无信息增益的验证事件。
- **T2 解释原因**：把该事件与最近代码变化、风险、失败签名和 Harness 规则关联起来。
- **T3 跨运行比较**：同一任务下比较三个 Harness 的验证路径、成功状态和成本。
- **T4 评估替代路径**：判断若在某点停止、缩小范围或先归因，最终 oracle 是否仍能满足。

可视化的主体应是“代码变化与验证事件耦合的时间线”，而不是总耗时柱状图。总览可以展示分布和排序，但论文的独特价值来自 drill-down：用户能从一次异常高成本运行下钻到具体重复命令、失败签名和当时的变更状态。

七、推荐实验设计

7.1 P0：36 条轨迹校准采集链路

使用现有 6 个本地受控任务，Codex、Maka、OpenCode 各运行 2 次。这个阶段不比较谁更好，只验证：

- 三个适配器是否都能捕获文件、命令、退出码和时间；
- canonical command 是否把 wrapper、workspace script 和直接 runner 调用正确合并；
- 工作树是否真正隔离；
- 相同任务的 full oracle 是否一致；
- 原始日志到统一 schema 是否可逆追溯；
- 可视化是否能正确显示预设的重复与非重复行为。

P0 失败时不得继续 P1。缺失事件若集中在某个 Harness，应先修适配器，不允许用模型补写不存在的轨迹。

7.2 P1 : 216 条平衡 pilot 轨迹

使用现有 24 个平衡任务包，每个 Harness 每任务运行 3 次。运行顺序按任务 block 使用 Latin square 或固定 seed 轮换，避免机器热状态和服务端负载总偏向某个 Harness。每次从相同 base commit 创建新工作树；依赖安装时间单列，主指标使用一致的 warm dependency cache。

P1 的主结果不是“冠军”，而是：

- 冗余类型的频率与成本分布；
- 不同项目类型、难度、风险和验证成本下的模式差异；
- Harness 间描述性差异与区间；
- 模型预标注相对人类裁决的准确性和不确定性；
- 用户借助可视化完成 T1/T2/T3 的时间与正确率。

统计单位是任务/项目。3 次重复用于估计模型与服务端随机性，不能当成 72 个独立任务。报告应按项目 -> 任务分层 bootstrap，并展示 median、P95 和区间；不能只给总体平均数。

7.3 P1.5 : 污染与外部效度

公开 benchmark 可能被模型见过。P1 至少加入或替换出 25% held-out Agent 任务，来源可以是：

- 模型快照之后创建、尚未公开修复的 maintainer issue；
- 合作者私有仓库中经过脱敏的缺陷任务；
- 在真实仓库固定 commit 上预注册生成、由 maintainer 审核的任务。

held-out 任务不能在同一批数据上调规则。Prompt、预算、canonicalization 和 label rubric 必须先在 tuning cohort 冻结。若拿不到 held-out，只能把结论限定为“公开 benchmark 上的轨迹差异”。

7.4 P2 : 270 条正式多语言主语料

P1 稳定后，建立一套规模适中的正式主语料：10 个 SWE-bench Verified Python 任务，加 20 个 SWE-PolyBench Verified 任务（Java、JavaScript、TypeScript、Python 各 5 个），每个任务由三个 Harness 各运行三次，共 270 条轨迹。它比 24 任务 pilot 增加了语言外部效度，又不会立即扩大到 720 次确认性运行。另抽 6 个 LoopsBench 任务做长程案例，不并入主效应统计。

7.5 P3：干预实验与强确认性研究

观察性研究完成后，再测试两种产品模式：

- 专家模式：显示完整证据和策略候选，由用户决定是否应用；
- 傻瓜模式：自动处理 exact/equivalent repeat 和未归因重跑，高风险或低置信度升级给用户。

干预实验至少比较 baseline 与 advisory。硬拦截只在低风险规则经过 pilot 后加入。所有“更快”必须同时通过 F2P、P2P、最终 full 和 mutation/seeded fault 闸门；resolved rate 的非劣界值可以沿用现有协议的 -5 个百分点，但样本不足时必须写“证据不足”，不能写“质量不变”。

7.6 用户研究

可视化论文还需要独立于 Agent 运行实验的用户研究。建议采用组内设计：参与者对同一批轨迹分别使用原始日志界面和 VerifyTrace，顺序平衡。任务是定位首次浪费点、选择失败类别、解释证据、决定是否停止或缩小测试。

主要指标可以是诊断正确率和完成时间，次指标是操作数、置信度校准和 NASA-TLX/简化认知负荷。不能只问“你喜欢哪个界面”。参与者优先招募真实使用 Coding Agent 的开发者；专家 Harness 研究者数量不足时，可作为定性访谈补充而不是唯一受试者。

八、工程与资源判断

8.1 本机

2026-08-15 fresh check：Apple Silicon arm64，32 GiB 内存，数据卷约 175 GiB 可用，Docker 28.1.1 的 server 为 Linux arm64。磁盘已经允许开始拉取部分容器，但需要先用 feasibility cohort 验证镜像架构、历史依赖和运行时间。

建议建立资源闸门：先运行一个 Python Verified task 和一个 JS/TS Multi-SWE task，记录镜像体积、构建时间、峰值磁盘和峰值内存。只有二者可复现，才批量准备 24 个任务。不要一次性拉取全部镜像后才发现架构不兼容。

8.2 DMIT VPS

VPS 的 x86_64 架构有利于兼容性，但 1 vCPU、1.9 GiB RAM、20 GiB 根盘不满足主实验。建议仅部署：

- 任务队列与状态页面；
- 压缩后的结构化事件和可视化静态站；

- 夜间调度通知；
- 不包含源码、凭据和原始敏感日志的汇总数据。

主实验需要本机 Docker、实验室服务器或按任务创建的 x86_64 云实例。结果必须记录硬件和缓存模式，不能把不同机器的绝对 wall time 直接混合。

8.3 成本控制

跨 Harness 研究的最大成本不是 token，而是无效环境。建议按以下顺序投入：

1. 完成两个边界项目的第二评审；
2. 跑 2 个公开任务的环境 probe；
3. 完成 36 条 P0 轨迹；
4. 人工检查事件缺失与归一化；
5. 冻结 schema、Prompt、模型和预算；
6. 才启动 216 条 P1。

模型使用必须选择能在三个 Harness 中通过同一官方或可验证接口调用的固定版本。第三方中转可以用于工程调试，但正式实验应记录供应商、模型快照和响应参数；无法证明模型身份时，论文应将其列为可复现性限制，并尽量用官方或开源权重模型复现一部分结果。

九、横纵交汇：真正的研究空位在哪里

9.1 历史如何塑造当前空位

代码 benchmark 最初只关心最终答案，所以测试被压缩成一个 pass/fail 裁判。SWE-bench 把测试带回开发过程，却仍以 resolved 为核心。SeaView 和 AgentGUI 开始展示轨迹，但它们面向通用动作、错误和长任务监督。**Same Task, Different Work** 已量化验证浪费，却没有提供面向代码变更与测试拓扑的可视诊断接口。

这条历史路径留下的空位很具体：现有 benchmark 有任务和 oracle，现有可视化有轨迹界面，现有测试研究有 regression test selection 和风险分层，但三者还没有被组合成一个“可重放、可裁决、可下钻、可干预”的验证行为研究系统。

VerifyTrace 的论文贡献不应写成“我们做了一个 Dashboard”。更合理的三点是：

1. 一个跨 Harness、带代码/测试因果上下文的验证事件模型与冗余 taxonomy；
2. 一个支持同任务跨运行比较和浪费起点诊断的可视分析系统；

3. 一组平衡、可重放且带安全 oracle 的任务与用户/系统评测，检验可视化是否提升诊断效率，并为后续干预提供证据。

9.2 最大机会

最大的机会不是证明“Agent 爱多测”，因为新论文已经给出强证据。机会是解释 Harness 如何塑造这种浪费，以及开发者能否在不阅读数万行日志的情况下找到可干预位置。

跨 Harness 比较必须固定模型，否则得到的是产品默认组合比较。固定模型后，Harness 的系统提示、工具封装、权限、上下文整理和停止规则才成为研究对象。可视化需要把这些不可见的系统差异映射到可观察事件，而不是猜测内部思维。

9.3 最大风险

最危险的失败方式有四种：

- 把“测试少”直接定义为好，导致系统通过漏测刷效率；
- 用 LLM judge 生成 ground truth，再拿它证明 LLM judge 有效；
- 同时改变模型、Harness、Prompt 和预算，最后无法归因；
- 先看结果再换任务，破坏预注册可信度。

现有 TestGuard 协议已经防住了后两项的一部分。新研究要继续坚持：效率必须服从正确性闸门，模型只做预标注，人类裁决模糊案例，任务与排除规则在结果前冻结。

9.4 三个未来剧本

最可能的剧本：P0/P1 形成一个可靠的跨 Harness 轨迹数据集和诊断界面，论文贡献落在 Agent 可视分析与实证 taxonomy。干预只作为设计含义，不在第一篇里宣称自动提速。

最危险的剧本：团队过早同时开发 Dashboard、策略引擎、三个适配器、自动学习和完整 benchmark，运行环境迟迟不稳定，最后只有几张模拟图和少量不可比日志。这会重复当前 vibe coding 的 Reward Hacking：功能看起来很多，研究证据没有增加。

最乐观的剧本：P1 发现跨 Harness 稳定的冗余模式，VerifyTrace 显著降低人类诊断时间；随后低风险 advisory 在随机实验中减少交互验证耗时，同时保持 oracle 非劣。届时项目可以同时产出可视化论文、软件工程实证论文和开源工具，但它们必须按证据成熟度分阶段发布。

十、最终建议与执行清单

10.1 立即采用

1. 不再新建任务集仓库，沿用 TestGuard 1.2.0 的 frozen manifest。
2. 把原“30 baseline + 30 shadow”改成 P0/P1/P2/P3 分阶段设计。
3. 第一篇只研究 baseline 下的跨 Harness 验证行为与可视诊断，不同时做三治理条件。
4. 使用 Codex、Maka、OpenCode；固定同一模型、任务、Prompt、权限和预算。
5. 在任务准入中新增验证可观测性 probe。
6. 统一事件 schema，不依赖隐藏 chain-of-thought。
7. 确定性规则标注明确重复，受约束模型预标注语境案例，人类裁决模糊/高风险案例。
8. 先完成 Sphinx、Express 的独立第二评审，再宣称 balanced pilot ready。

10.2 暂不采用

- 不直接把 SWE-bench leaderboard 当成实验任务设计；
- 不在第一轮加入 Pi、Hermes 或更多 Harness；
- 不在第一轮同时比较多个模型；
- 不直接启动 48 任务 × 3 Harness × 5 重复；
- 不把 DMIT VPS 当容器执行节点；
- 不让系统根据几次成功自动改写高风险测试策略；
- 不用“蠢测试”作为最终论文术语和标签。

10.3 下一个可验证检查点

下一个检查点不是画 UI，而是形成一个最小数据闭环：

- 1 个受控任务
- × Codex / Maka / OpenCode
- 3 条统一 schema 轨迹
- 自动重复检测
- 模型结构化预标注
- 人工裁决
- 同任务三列时间线原型

这个闭环跑通后，才能知道事件采集、可视编码和标注是否站得住。它比再写十页产品规划更能推动研究。

十一、信息来源

外部论文与数据集

1. Jimenez et al., “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?”, 2023/2024. <https://arxiv.org/abs/2310.06770>
2. OpenAI, “Introducing SWE-bench Verified”, 2024. <https://openai.com/index/introducing-swe-bench-verified/>
3. SWE-bench official repository. <https://github.com/SWE-bench/SWE-bench>
4. SWE-bench Verified dataset. https://huggingface.co/datasets/SWE-bench/SWE-bench_Verified
5. Multi-SWE-bench dataset. <https://huggingface.co/datasets/ByteDance-Seed/Multi-SWE-bench>
6. Multi-SWE-bench Flash dataset. <https://huggingface.co/datasets/ByteDance-Seed/Multi-SWE-bench-flash>
7. Multi-SWE-bench, “A Multilingual Benchmark for Issue Resolving”, DOI: 10.52202/085713-2111. <https://doi.org/10.52202/085713-2111>
8. “Multi-SWE-bench: A Multilingual Benchmark for Issue Resolving.” <https://arxiv.org/abs/2504.02605>
9. “SWT-Bench: Testing and Validating Real-World Bug-Fixes with Code Agents.” <https://arxiv.org/abs/2406.12952>
10. “SWE-smith: Scaling Data for Software Engineering Agents.” <https://arxiv.org/abs/2504.21798>
11. “SWE-bench Live: Can AI Agents Resolve Issue Reports from the Future?” <https://arxiv.org/abs/2505.23419>
12. “SWE-PolyBench: A Multi-Language Benchmark for Repository-Level Issue Resolution.” <https://arxiv.org/abs/2504.08703>
13. SWE-PolyBench official repository. <https://github.com/amazon-science/SWE-PolyBench>
14. “SWE-Bench+: Enhanced Coding Benchmark with Test Adequacy and Contamination Audit.” <https://arxiv.org/abs/2410.06992>
15. “UTBoost: Rigorous Evaluation of Coding Agents on SWE-Bench.” <https://arxiv.org/abs/2506.09289>
16. “SeaView: Software Engineering Agent Visual Interface for Enhanced Workflow.” <https://arxiv.org/abs/2504.08696>

17. “Understanding Automated Program Repair Agents Through the Lens of Traceability: An Empirical Study.” <https://arxiv.org/abs/2506.08311>
18. “AgentGUI: An Interface for Observing and Steering Long-Running AI Agents.” <https://arxiv.org/abs/2607.26300>
19. “LoopsBench: From Harness Engineering to Loop Engineering in Coding Agent Evaluation.” <https://arxiv.org/abs/2608.00267>
20. “Same Task, Different Work: Prompt-Induced Waste in Coding Agents.” <https://arxiv.org/abs/2608.01347>
21. “Coding Agents as Test-Suite Auditors: Finding What Official Suites Miss While Approaching What They Catch.” <https://arxiv.org/abs/2608.01715>
22. “Diagnosis Before Recovery: Turning Agent Failures into Selective Self-Correction.” <https://arxiv.org/abs/2608.11772>
23. Harbor framework. <https://github.com/harbor-framework/harbor>
24. Harbor Agent Trajectory Interchange Format documentation. <https://github.com/harbor-framework/harbor/blob/main/docs/content/docs/agents/trajectory-format.mdx>

本地一手证据

1. TestGuard benchmark protocol v1.2.0 ([benchmarks/PROTOCOL.md](#) , local research source).
2. TestGuard machine-readable study ([benchmarks/study.json](#) , local research source).
3. TestGuard frozen task manifest ([benchmarks/tasks.json](#) , local research source).
4. TestGuard credibility levels ([benchmarks/CREDIBILITY.md](#) , local research source).
5. TestGuard project type review packet ([benchmarks/review-packet.md](#) , local research source).
6. [AI Coding Agent Test Strategy experiment plan](#).
7. DMIT VPS current handoff (local infrastructure record, not included in this repository).

所有网络来源访问时间为 2026-08-15。部分站点在当前网络下间歇性断开；关键数据集 revision、任务数量与本地状态以固定 manifest 和 fresh 本地检查为准，未能在线复核的产品宣传数字没有用于实验结论。

十二、方法论说明

本报告使用横纵分析法：纵向追踪代码评测从函数 pass/fail、真实 issue 到 Agent loop 与轨迹分析的演进；横向比较当前可用任务来源的 oracle、可重放性、多语言覆盖、污染与成本；最后把两条轴交叉为适合 VerifyTrace 可视化论文的分阶段任务集设计。